
FITCORE

Personalized Fitness Tracker

Database Management System — Project Report

[GitHub Link](#)

Team Members

No.	Name	Enrollment
1	Dev Kansara	AU2340222
2	Nirmam Parikh	AU2340253
3	Parin Patel	AU2340243
4	Utsav Panchal	AU2340122

March 30, 2026

Contents

1	Abstract	3
2	Introduction	4
2.1	Objectives	4
2.2	Technology Stack	4
3	ER Diagram & Database Design	5
3.1	Entity-Relationship Diagram	5
3.2	Entities and Attributes	6
3.3	Relationships	6
4	Schema & Table Definitions	7
4.1	CREATE TABLE Statements	7
5	Normalisation	10
5.1	First Normal Form (1NF)	10
5.2	Second Normal Form (2NF)	10
5.3	Third Normal Form (3NF)	10
6	SQL Queries	11
6.1	Query 1 — Top Performing Users by Calories Burned	11
6.2	Query 2 — Users Who Missed Workouts	11
6.3	Query 3 — Nutrition Deficit or Surplus Per User	12
6.4	Query 4 — Most Popular Workout Types	12
6.5	Query 5 — User Goal Progress Overview	13
6.6	Query 6 — Average Sleep Per User with Status	13
6.7	Query 7 — Calories Burned vs Consumed	14
7	Database Views	15
7.1	View 1 — Weekly Progress Report	15

7.2	View 2 — Average Calories Per Workout Type	16
7.3	View 3 — Sleep Quality Summary	16
7.4	View 4 — User Nutrition Summary	17
7.5	View 5 — Latest Weather Per City	17
8	PL/SQL Cursors	19
8.1	Cursor 1 — Daily Recommendation Generator	19
8.2	Cursor 2 — Goal Progress Tracker	24
8.3	Cursor 3 — Weekly Workout Streak Counter	26
9	Frontend Description	28
9.1	Architecture Overview	28
9.2	Role-Based Access Control	28
9.3	REST API Endpoint Reference	29
10	Screenshots	30
11	Conclusion & Future Scope	33
11.1	Conclusion	33
11.2	Future Scope	33
	References	34

Chapter 1

Abstract

The **Personalized Fitness Tracker** is a full-stack database-driven application designed to help individuals monitor and improve their health and fitness routines. Built on **Oracle Database XE**, the system centralises data across seven core domains: user profiles, workout logs, meal nutrition, sleep patterns, fitness goals, real-time weather conditions, and personalised health recommendations.

The project demonstrates practical application of relational database concepts including entity-relationship modelling, normalisation up to 3NF, PL/SQL explicit cursors, sequences, five database views, and seven complex analytical queries. A RESTful API layer built with **Python Flask** exposes all database operations to a responsive **HTML/CSS/JavaScript** frontend supporting two distinct roles: *User* (personal dashboard and personal data entry) and *Coach* (full visibility over all members, analytics, and recommendations).

Integration with the **OpenWeatherMap API** fetches live weather data per user location, which the cursor logic uses to generate context-aware daily recommendations.

Chapter 2

Introduction

2.1 Objectives

- Design a normalised relational schema covering all aspects of fitness data.
- Implement PL/SQL explicit cursors for intelligent, data-driven recommendation generation.
- Create five database views to support analytical reporting.
- Expose all database operations through a Flask REST API.
- Build a role-based frontend where users see only their own data and coaches see all members.
- Integrate real-time weather data per user location to contextualise recommendations.

2.2 Technology Stack

Layer	Technology
Database	Oracle Database 21c XE (XEPDB1), PL/SQL
Backend	Python 3.11, Flask 3.x, python-oracledb (thin mode)
Frontend	HTML5, CSS3 (custom design system), Vanilla JavaScript
External API	OpenWeatherMap Current Weather Data API
Version Control	Git / GitHub

Table 2.1: Project technology stack

Chapter 3

ER Diagram & Database Design

3.1 Entity-Relationship Diagram

The ER diagram below illustrates all seven entities and their relationships. **USERS** is the central entity, holding one-to-many (1:N) relationships with all activity tables. The **WEATHER** table is linked to **USERS** logically via the **location** attribute rather than a foreign key, since weather is city-level data shared across users.

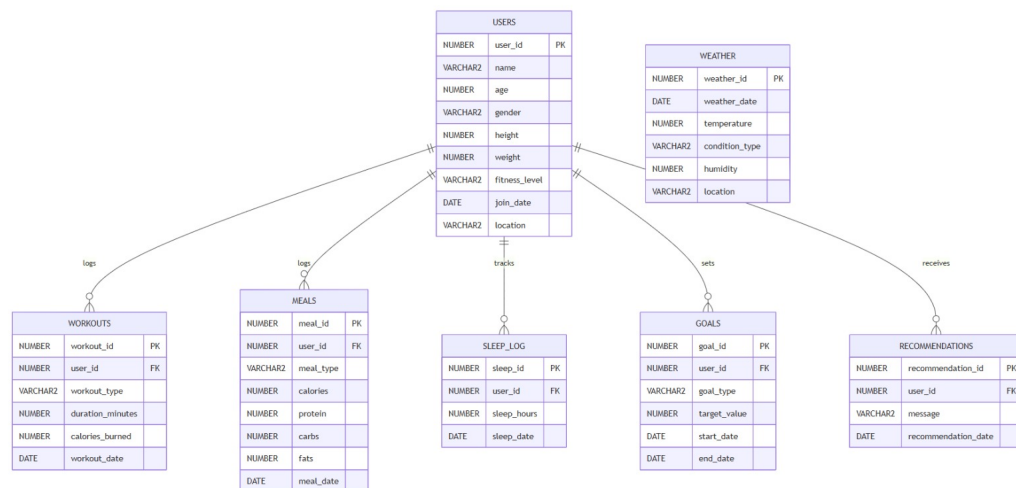


Figure 3.1: Entity-Relationship Diagram — Personalized Fitness Tracker

3.2 Entities and Attributes

Entity	Primary Key	Attributes
USERS	user_id	name, age, gender, height, weight, fitness_level, location, join_date
WORKOUTS	workout_id	user_id (FK), workout_type, duration_minutes, calories_burned, workout_date
MEALS	meal_id	user_id (FK), meal_type, calories, protein, carbs, fats, meal_date
SLEEP_LOG	sleep_id	user_id (FK), sleep_hours, sleep_date
GOALS	goal_id	user_id (FK), goal_type, target_value, start_date, end_date
RECOMMENDATIONS	recommendation_id	user_id (FK), message, recommendation_date
WEATHER	weather_id	location, temperature, condition_type, humidity, weather_date

Table 3.1: Entity summary with primary and foreign keys

3.3 Relationships

- **USERS — WORKOUTS (1:N)**: One user logs many workouts. Each workout references `user_id` as a foreign key.
- **USERS — MEALS (1:N)**: One user logs many meals. Nutritional macros (protein, carbs, fats) are stored atomically per meal.
- **USERS — SLEEP_LOG (1:N)**: One user has many sleep entries, one record per night.
- **USERS — GOALS (1:N)**: One user can set multiple fitness goals with start and end date ranges.
- **USERS — RECOMMENDATIONS (1:N)**: One user receives many recommendations inserted by the PL/SQL cursor block using `rec_seq`.
- **WEATHER — USERS (logical)**: `WEATHER` is not foreign-keyed to `USERS`. The cursor matches `UPPER(location)` at runtime to fetch city-level weather per user.

Chapter 4

Schema & Table Definitions

All tables are created under the **SYSTEM** schema on XEPDB1. The complete DDL is from `create_tables.sql`.

4.1 CREATE TABLE Statements

```
1 CREATE TABLE USERS (  
2     user_id      NUMBER PRIMARY KEY,  
3     name         VARCHAR2(50) NOT NULL,  
4     age          NUMBER,  
5     gender       VARCHAR2(10),  
6     height       NUMBER,  
7     weight       NUMBER,  
8     fitness_level VARCHAR2(20),  
9     join_date    DATE,  
10    location     VARCHAR2(100)  
11 );
```

Listing 4.1: USERS table (create_tables.sql)

```
1 CREATE TABLE WEATHER (  
2     weather_id    NUMBER PRIMARY KEY,  
3     weather_date  DATE,  
4     temperature   NUMBER,  
5     condition_type VARCHAR2(50),  
6     humidity      NUMBER,  
7     location      VARCHAR2(100)  
8 );
```

Listing 4.2: WEATHER table (create_tables.sql)

```
1 CREATE TABLE WORKOUTS (  
2     workout_id    NUMBER PRIMARY KEY,  
3     user_id       NUMBER,
```



```
4      workout_type      VARCHAR2(50),
5      duration_minutes  NUMBER,
6      calories_burned   NUMBER,
7      workout_date      DATE,
8      FOREIGN KEY (user_id) REFERENCES USERS(user_id)
9  );
```

Listing 4.3: WORKOUTS table (create_tables.sql)

```
1  CREATE TABLE MEALS (
2      meal_id    NUMBER PRIMARY KEY,
3      user_id    NUMBER,
4      meal_type  VARCHAR2(20),
5      calories   NUMBER,
6      protein    NUMBER,
7      carbs      NUMBER,
8      fats       NUMBER,
9      meal_date  DATE,
10     FOREIGN KEY (user_id) REFERENCES USERS(user_id)
11 );
```

Listing 4.4: MEALS table (create_tables.sql)

```
1  CREATE TABLE SLEEP_LOG (
2      sleep_id    NUMBER PRIMARY KEY,
3      user_id     NUMBER,
4      sleep_hours  NUMBER,
5      sleep_date   DATE,
6      FOREIGN KEY (user_id) REFERENCES USERS(user_id)
7  );
```

Listing 4.5: SLEEP_LOG table (create_tables.sql)

```
1  CREATE TABLE GOALS (
2      goal_id      NUMBER PRIMARY KEY,
3      user_id      NUMBER,
4      goal_type     VARCHAR2(50),
5      target_value  NUMBER,
6      start_date    DATE,
7      end_date      DATE,
8      FOREIGN KEY (user_id) REFERENCES USERS(user_id)
9  );
```

Listing 4.6: GOALS table (create_tables.sql)

```
1 CREATE TABLE RECOMMENDATIONS (  
2     recommendation_id    NUMBER PRIMARY KEY,  
3     user_id              NUMBER,  
4     message              VARCHAR2(500),  
5     recommendation_date  DATE,  
6     FOREIGN KEY (user_id) REFERENCES USERS(user_id)  
7 );
```

Listing 4.7: RECOMMENDATIONS table (create_tables.sql)

```
1 CREATE SEQUENCE weather_seq START WITH 1 INCREMENT BY 1;  
2 CREATE SEQUENCE rec_seq      START WITH 1 INCREMENT BY 1;
```

Listing 4.8: Sequences (create_tables.sql)

Chapter 5

Normalisation

5.1 First Normal Form (1NF)

All tables satisfy 1NF:

- Every column holds **atomic** values. Nutritional data is split into separate **protein**, **carbs**, and **fats** columns rather than a single composite field.
- Each row is uniquely identified by a single-column primary key.

5.2 Second Normal Form (2NF)

All tables satisfy 2NF:

- All primary keys are single-column (**NUMBER**), so composite keys do not exist.
- Since there are no composite keys, partial functional dependency cannot arise by definition.

5.3 Third Normal Form (3NF)

All tables satisfy 3NF — no transitive dependencies exist:

- In **WORKOUTS**, **calories_burned** depends on **workout_id**, not on **workout_type**. The same workout type logged on different days by different users can yield different calorie values.
- In **USERS**, **location** is a direct user attribute, not derivable from any other non-key column.
- **WEATHER** is stored independently of **USERS**. Matching on **location** at query time means multiple users in the same city share a single weather record without data duplication.

Chapter 6

SQL Queries

All seven queries below are taken directly from `queries.sql`.

6.1 Query 1 — Top Performing Users by Calories Burned

Groups all workout sessions per user and ranks them by total calories burned. Uses `INNER JOIN`, `GROUP BY`, and `ORDER BY`.

```
1  SELECT
2      u.name ,
3      u.fitness_level ,
4      SUM(w.calories_burned) AS total_calories_burned ,
5      COUNT(w.workout_id)   AS total_workouts
6  FROM USERS u
7  JOIN WORKOUTS w ON u.user_id = w.user_id
8  GROUP BY u.name, u.fitness_level
9  ORDER BY total_calories_burned DESC;
```

Listing 6.1: Top performers by calories burned (`queries.sql`)

6.2 Query 2 — Users Who Missed Workouts

Detects users who have not logged a workout in more than 2 days. Uses `LEFT JOIN`, `HAVING`, and Oracle date arithmetic with `TRUNC(SYSDATE)`.

```
1  SELECT
2      u.name ,
3      u.fitness_level ,
4      u.location ,
5      MAX(w.workout_date) AS last_workout_date ,
6      TRUNC(SYSDATE) - TRUNC(MAX(w.workout_date)) AS
7          days_since_last_workout
8  FROM USERS u
```

```
8 LEFT JOIN WORKOUTS w ON u.user_id = w.user_id
9 GROUP BY u.user_id, u.name, u.fitness_level, u.location
10 HAVING TRUNC(SYSDATE) - TRUNC(MAX(w.workout_date)) > 2
11      OR MAX(w.workout_date) IS NULL
12 ORDER BY days_since_last_workout DESC;
```

Listing 6.2: Missed workout detection (queries.sql)

6.3 Query 3 — Nutrition Deficit or Surplus Per User

Classifies each user's average daily calorie intake against a 2000 kcal standard using a CASE expression.

```
1 SELECT
2     u.name,
3     ROUND(AVG(m.calories), 2) AS avg_daily_calories,
4     2000 - ROUND(AVG(m.calories), 2) AS calorie_deficit_surplus,
5     CASE
6         WHEN AVG(m.calories) < 1500 THEN 'Under Eating'
7         WHEN AVG(m.calories) > 2500 THEN 'Over Eating'
8         ELSE 'On Track'
9     END AS nutrition_status
10 FROM USERS u
11 JOIN MEALS m ON u.user_id = m.user_id
12 GROUP BY u.name
13 ORDER BY avg_daily_calories DESC;
```

Listing 6.3: Nutrition deficit/surplus with CASE (queries.sql)

6.4 Query 4 — Most Popular Workout Types

Aggregates all workout sessions by type, showing frequency, average duration, and average calories burned.

```
1 SELECT
2     workout_type,
3     COUNT(*) AS frequency,
4     ROUND(AVG(duration_minutes), 2) AS avg_duration,
5     ROUND(AVG(calories_burned), 2) AS avg_calories
6 FROM WORKOUTS
7 GROUP BY workout_type
```

```
8 ORDER BY frequency DESC;
```

Listing 6.4: Popular workout types (queries.sql)

6.5 Query 5 — User Goal Progress Overview

Shows remaining days and deadline status for every active goal using JOIN and a CASE expression.

```
1 SELECT
2     u.name ,
3     g.goal_type ,
4     g.target_value ,
5     g.start_date ,
6     g.end_date ,
7     TRUNC(g.end_date - SYSDATE) AS days_remaining ,
8     CASE
9         WHEN SYSDATE > g.end_date THEN 'Expired'
10        WHEN TRUNC(g.end_date - SYSDATE) <= 7 THEN 'Deadline Near'
11        ELSE 'In Progress'
12    END AS goal_status
13 FROM USERS u
14 JOIN GOALS g ON u.user_id = g.user_id
15 ORDER BY days_remaining;
```

Listing 6.5: Goal progress overview (queries.sql)

6.6 Query 6 — Average Sleep Per User with Status

Ranks users by average nightly sleep and classifies quality as Excellent, Adequate, or Poor.

```
1 SELECT
2     u.name ,
3     ROUND(AVG(s.sleep_hours), 2) AS avg_sleep ,
4     CASE
5         WHEN AVG(s.sleep_hours) >= 8 THEN 'Excellent'
6         WHEN AVG(s.sleep_hours) >= 6 THEN 'Adequate'
7         ELSE 'Poor - Needs Improvement'
8     END AS sleep_status
9 FROM USERS u
```

```
10 JOIN SLEEP_LOG s ON u.user_id = s.user_id
11 GROUP BY u.name
12 ORDER BY avg_sleep DESC;
```

Listing 6.6: Sleep quality ranking (queries.sql)

6.7 Query 7 — Calories Burned vs Consumed

Computes net calorie balance per user using dual LEFT JOINS and classifies the result as Surplus or Deficit.

```
1  SELECT
2      u.name ,
3      NVL(SUM(w.calories_burned), 0) AS total_burned ,
4      NVL(SUM(m.calories), 0) AS total_consumed ,
5      NVL(SUM(m.calories), 0) -
6      NVL(SUM(w.calories_burned), 0) AS net_calorie_balance ,
7      CASE
8          WHEN NVL(SUM(m.calories), 0) -
9              NVL(SUM(w.calories_burned), 0) > 0
10             THEN 'Calorie Surplus'
11             ELSE 'Calorie Deficit'
12         END AS balance_status
13 FROM USERS u
14 LEFT JOIN WORKOUTS w ON u.user_id = w.user_id
15 LEFT JOIN MEALS m ON u.user_id = m.user_id
16 GROUP BY u.name
17 ORDER BY net_calorie_balance DESC;
```

Listing 6.7: Calorie balance — burned vs consumed (queries.sql)

Chapter 7

Database Views

All five views below are taken directly from `views.sql`. They are consumed by the Flask API endpoints `/api/view/{name}` and rendered in the Coach Analytics tab on the frontend.

7.1 View 1 — Weekly Progress Report

Summarises the last 7 days of activity per user: workout count, total workout minutes, calories burned, calories consumed, and average sleep. Uses correlated subqueries for each metric.

```
1 CREATE OR REPLACE VIEW Weekly_Progress AS
2 SELECT
3     u.user_id,
4     u.name,
5     u.location,
6     (SELECT COUNT(*)
7      FROM WORKOUTS w
8      WHERE w.user_id = u.user_id
9      AND w.workout_date >= SYSDATE - 7)          AS
10         total_workouts,
11     (SELECT NVL(SUM(w.duration_minutes), 0)
12      FROM WORKOUTS w
13      WHERE w.user_id = u.user_id
14      AND w.workout_date >= SYSDATE - 7)          AS
15         total_workout_mins,
16     (SELECT NVL(SUM(w.calories_burned), 0)
17      FROM WORKOUTS w
18      WHERE w.user_id = u.user_id
19      AND w.workout_date >= SYSDATE - 7)          AS
20         total_calories_burned,
21     (SELECT NVL(SUM(m.calories), 0)
22      FROM MEALS m
23      WHERE m.user_id = u.user_id
24      AND m.consumption_date >= SYSDATE - 7)      AS
25         total_calories_consumed,
```



```

20     WHERE m.user_id = u.user_id
21     AND m.meal_date >= SYSDATE - 7)          AS
        total_calories_consumed,
22     (SELECT ROUND(NVL(AVG(s.sleep_hours), 0), 2)
23     FROM SLEEP_LOG s
24     WHERE s.user_id = u.user_id
25     AND s.sleep_date >= SYSDATE - 7)          AS
        avg_sleep_hours
26 FROM USERS u;

```

Listing 7.1: Weekly_Progress view (views.sql)

7.2 View 2 — Average Calories Per Workout Type

Groups all workout sessions by type, computing total sessions, average duration, average calories, and total calories burned.

```

1  CREATE OR REPLACE VIEW Avg_Calories_Per_Workout AS
2  SELECT
3      workout_type,
4      COUNT(*)                AS total_sessions,
5      ROUND(AVG(duration_minutes), 2) AS avg_duration_mins,
6      ROUND(AVG(calories_burned), 2) AS avg_calories_burned,
7      SUM(calories_burned)        AS total_calories_burned
8  FROM WORKOUTS
9  GROUP BY workout_type
10 ORDER BY avg_calories_burned DESC;

```

Listing 7.2: Avg_Calories_Per_Workout view (views.sql)

7.3 View 3 — Sleep Quality Summary

Per-user sleep analytics: average, minimum, and maximum sleep hours, plus counts of poor nights (<6 hrs), average nights (6–8 hrs), and good nights (≥8 hrs).

```

1  CREATE OR REPLACE VIEW Sleep_Quality_Summary AS
2  SELECT
3      u.user_id,
4      u.name,
5      ROUND(AVG(s.sleep_hours), 2) AS avg_sleep_hours,
6      MIN(s.sleep_hours)          AS min_sleep_hours,

```

```
7      MAX(s.sleep_hours)          AS max_sleep_hours ,
8      SUM(CASE WHEN s.sleep_hours < 6   THEN 1 ELSE 0 END) AS
      poor_sleep_days ,
9      SUM(CASE WHEN s.sleep_hours >= 6
10          AND s.sleep_hours < 8   THEN 1 ELSE 0 END) AS
      average_sleep_days ,
11      SUM(CASE WHEN s.sleep_hours >= 8   THEN 1 ELSE 0 END) AS
      good_sleep_days
12 FROM USERS u
13 LEFT JOIN SLEEP_LOG s ON u.user_id = s.user_id
14 GROUP BY u.user_id, u.name;
```

Listing 7.3: Sleep_Quality_Summary view (views.sql)

7.4 View 4 — User Nutrition Summary

Per-user nutritional overview: average daily calories and average macro breakdown (protein, carbs, fats), plus total calories across all logged meals.

```
1 CREATE OR REPLACE VIEW Nutrition_Summary AS
2 SELECT
3     u.user_id ,
4     u.name ,
5     ROUND(AVG(m.calories), 2) AS avg_daily_calories ,
6     ROUND(AVG(m.protein), 2) AS avg_protein ,
7     ROUND(AVG(m.carbs), 2) AS avg_carbs ,
8     ROUND(AVG(m.fats), 2) AS avg_fats ,
9     SUM(m.calories) AS total_calories
10 FROM USERS u
11 LEFT JOIN MEALS m ON u.user_id = m.user_id
12 GROUP BY u.user_id, u.name;
```

Listing 7.4: Nutrition_Summary view (views.sql)

7.5 View 5 — Latest Weather Per City

Returns only the most recent weather record per city using a correlated subquery on `MAX(weather_date)`, preventing duplicate rows from multiple daily API fetches.

```
1 CREATE OR REPLACE VIEW Latest_Weather AS
2 SELECT
```

```
3      location,
4      temperature,
5      condition_type,
6      humidity,
7      weather_date
8 FROM WEATHER w1
9 WHERE weather_date = (
10     SELECT MAX(weather_date)
11     FROM WEATHER w2
12     WHERE w2.location = w1.location
13 );
```

Listing 7.5: Latest_Weather view (views.sql)

Chapter 8

PL/SQL Cursors

8.1 Cursor 1 — Daily Recommendation Generator

This is the core PL/SQL block of the project. It uses an **explicit cursor** to iterate over every user and, for each one, fetches four additional data points using nested `SELECT INTO` statements with `NO_DATA_FOUND` exception handling: today's weather for their city, yesterday's workout, yesterday's meal totals, and last night's sleep. It then builds a personalised `VARCHAR2(500)` message using cascaded `IF/ELSIF` logic and inserts it into the `RECOMMENDATIONS` table via `rec_seq.NEXTVAL`.

```
1  SET SERVEROUTPUT ON;
2
3  DECLARE
4      CURSOR user_cur IS
5          SELECT user_id, name, fitness_level, location
6              FROM USERS;
7
8      v_user_id      USERS.user_id%TYPE;
9      v_name         USERS.name%TYPE;
10     v_fitness_level USERS.fitness_level%TYPE;
11     v_location      USERS.location%TYPE;
12     v_temp          WEATHER.temperature%TYPE;
13     v_condition     WEATHER.condition_type%TYPE;
14     v_humidity      WEATHER.humidity%TYPE;
15     v_workout_type   WORKOUTS.workout_type%TYPE;
16     v_duration       WORKOUTS.duration_minutes%TYPE;
17     v_calories_burned WORKOUTS.calories_burned%TYPE;
18     v_total_calories NUMBER;
19     v_total_protein  NUMBER;
20     v_total_carbs    NUMBER;
21     v_sleep_hours    SLEEP_LOG.sleep_hours%TYPE;
22     v_goal_type      GOALS.goal_type%TYPE;
23     v_message        VARCHAR2(500);
```

```
24
25 BEGIN
26     OPEN user_cur;
27     LOOP
28         FETCH user_cur INTO v_user_id, v_name, v_fitness_level,
29                             v_location;
30         EXIT WHEN user_cur%NOTFOUND;
31
32         -- Fetch today's weather for user's city
33         BEGIN
34             SELECT temperature, condition_type, humidity
35             INTO v_temp, v_condition, v_humidity
36             FROM WEATHER
37             WHERE UPPER(location) = UPPER(v_location)
38             AND TRUNC(weather_date) = TRUNC(SYSDATE)
39             AND ROWNUM = 1;
40         EXCEPTION
41             WHEN NO_DATA_FOUND THEN
42                 v_temp := NULL; v_condition := 'Unknown';
43                 v_humidity := NULL;
44         END;
45
46         -- Fetch yesterday's workout
47         BEGIN
48             SELECT workout_type, duration_minutes, calories_burned
49             INTO v_workout_type, v_duration, v_calories_burned
50             FROM WORKOUTS
51             WHERE user_id = v_user_id
52             AND TRUNC(workout_date) = TRUNC(SYSDATE - 1)
53             AND ROWNUM = 1;
54         EXCEPTION
55             WHEN NO_DATA_FOUND THEN
56                 v_workout_type := NULL; v_duration := 0;
57                 v_calories_burned := 0;
58         END;
59
60         -- Fetch yesterday's meal totals
61         BEGIN
62             SELECT NVL(SUM(calories), 0),
63                   NVL(SUM(protein), 0),
64                   NVL(SUM(carbs), 0)
```

```
62         INTO v_total_calories, v_total_protein, v_total_carbs
63         FROM MEALS
64         WHERE user_id = v_user_id
65         AND TRUNC(meal_date) = TRUNC(SYSDATE - 1);
66     EXCEPTION
67         WHEN NO_DATA_FOUND THEN
68         v_total_calories := 0; v_total_protein := 0;
69         v_total_carbs := 0;
70     END;
71
72     -- Fetch last night's sleep
73     BEGIN
74         SELECT sleep_hours INTO v_sleep_hours
75         FROM SLEEP_LOG
76         WHERE user_id = v_user_id
77         AND TRUNC(sleep_date) = TRUNC(SYSDATE - 1);
78     EXCEPTION
79         WHEN NO_DATA_FOUND THEN v_sleep_hours := 0;
80     END;
81
82     -- Fetch current goal
83     BEGIN
84         SELECT goal_type INTO v_goal_type
85         FROM GOALS
86         WHERE user_id = v_user_id AND ROWNUM = 1;
87     EXCEPTION
88         WHEN NO_DATA_FOUND THEN v_goal_type := 'General
89         Fitness';
90     END;
91
92     -- Build personalised recommendation message
93     v_message := 'Hey ' || v_name || '! ';
94
95     IF v_sleep_hours = 0 THEN
96         v_message := v_message || 'No sleep data found for
97         yesterday. ';
98     ELSIF v_sleep_hours < 6 THEN
99         v_message := v_message || 'You only slept ' ||
100         v_sleep_hours
101         || ' hrs - rest is critical for recovery! '
102         || 'Consider a light yoga or stretching
```

```

        session today. ';
99     ELSIF v_sleep_hours < 7.5 THEN
100         v_message := v_message || 'You slept ' ||
            v_sleep_hours
101             || ' hrs - decent but aim for 8hrs. ';
102     ELSE
103         v_message := v_message || 'Great sleep of ' ||
            v_sleep_hours
104             || ' hrs! Your body is well recovered. ';
105     END IF;
106
107     IF v_condition IN ('Rain','Drizzle','Thunderstorm') THEN
108         v_message := v_message || 'It is ' || v_condition ||
            ' in '
109             || v_location || ' today - skip outdoor
            workouts. ';
110     ELSIF v_temp > 38 THEN
111         v_message := v_message || 'Very hot (' || v_temp ||
            'C) in '
112             || v_location || ' - avoid outdoor
            exercise! ';
113     ELSIF v_temp > 30 THEN
114         v_message := v_message || 'Warm (' || v_temp || 'C)
            in '
115             || v_location || ' - workout early morning
            or evening. ';
116     ELSIF v_temp >= 20 THEN
117         v_message := v_message || 'Perfect weather (' ||
            v_temp || 'C) in '
118             || v_location || ' for an outdoor workout!
            ';
119     ELSE
120         v_message := v_message || 'Cool weather (' || v_temp
            || 'C) in '
121             || v_location || ' - great for a brisk run
            or cycling! ';
122     END IF;
123
124     IF v_workout_type IS NULL THEN
125         v_message := v_message || 'No workout logged
            yesterday - do not skip! ';

```

```
126     ELSIF v_duration >= 60 THEN
127         v_message := v_message || 'Excellent ' || v_duration
128             || ' min '
129             || v_workout_type || ' yesterday! Consider
130                 active recovery. ';
131     ELSIF v_duration >= 30 THEN
132         v_message := v_message || 'Good ' || v_duration || '
133             min '
134             || v_workout_type || ' yesterday! Push
135                 harder today. ';
136     ELSE
137         v_message := v_message || 'Only ' || v_duration
138             || ' min yesterday - aim for at least 30
139                 mins! ';
140     END IF;
141
142     IF v_total_calories = 0 THEN
143         v_message := v_message || 'No meals logged - track
144             your food! ';
145     ELSIF v_total_calories > 2500 THEN
146         v_message := v_message || 'High intake (' ||
147             v_total_calories
148             || ' kcal) yesterday - eat clean today. ';
149     ELSIF v_total_calories < 1200 THEN
150         v_message := v_message || 'Very low (' ||
151             v_total_calories
152             || ' kcal) yesterday - fuel your body! ';
153     ELSE
154         v_message := v_message || 'Good calorie intake ('
155             || v_total_calories || ' kcal) yesterday. ';
156     END IF;
157
158     IF v_goal_type = 'Weight Loss' THEN
159         v_message := v_message || 'Focus on cardio and
160             calorie deficit!';
161     ELSIF v_goal_type = 'Muscle Gain' THEN
162         v_message := v_message || 'Hit the weights and keep
163             protein high!';
164     ELSIF v_goal_type = 'Run Distance' THEN
165         v_message := v_message || 'Add 5-10 mins to your run
166             today!';
```



```

156     ELSE
157         v_message := v_message || 'Stay consistent and keep
            tracking!';
158     END IF;
159
160     INSERT INTO RECOMMENDATIONS
161         (recommendation_id, user_id, message,
            recommendation_date)
162     VALUES (rec_seq.NEXTVAL, v_user_id, v_message, SYSDATE);
163
164     DBMS_OUTPUT.PUT_LINE('--- Recommendation for ' || v_name
        || ' ---');
165     DBMS_OUTPUT.PUT_LINE(v_message);
166 END LOOP;
167 CLOSE user_cur;
168 COMMIT;
169 DBMS_OUTPUT.PUT_LINE('All recommendations generated and
    saved!');
170 EXCEPTION
171     WHEN OTHERS THEN
172         DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
173         ROLLBACK;
174 END;
175 /

```

Listing 8.1: Cursor 1 — Daily recommendation generator (cursors.sql)

8.2 Cursor 2 — Goal Progress Tracker

Iterates over all user goals, computes what percentage of the goal's time window has elapsed using date arithmetic, and prints a status message (*JUST STARTED*, *GOOD START*, *HALFWAY*, *ALMOST THERE*, or *EXPIRED*).

```

1 DECLARE
2     CURSOR goal_cur IS
3         SELECT u.name, g.goal_type, g.target_value,
4             g.start_date, g.end_date
5         FROM USERS u
6         JOIN GOALS g ON u.user_id = g.user_id;
7
8     v_name          USERS.name%TYPE;

```

```
9      v_goal_type      GOALS.goal_type%TYPE;
10     v_target         GOALS.target_value%TYPE;
11     v_start          GOALS.start_date%TYPE;
12     v_end            GOALS.end_date%TYPE;
13     v_days_total     NUMBER;
14     v_days_done      NUMBER;
15     v_progress_pct   NUMBER;
16     v_status         VARCHAR2(100);
17
18 BEGIN
19     OPEN goal_cur;
20     LOOP
21         FETCH goal_cur INTO v_name, v_goal_type, v_target,
22             v_start, v_end;
23
24         EXIT WHEN goal_cur%NOTFOUND;
25
26         v_days_total      := TRUNC(v_end - v_start);
27         v_days_done       := TRUNC(SYSDATE - v_start);
28         v_progress_pct    := ROUND((v_days_done /
29             NULLIF(v_days_total, 0)) * 100, 2);
30
31         IF      SYSDATE > v_end                THEN v_status := 'EXPIRED
32             - Goal deadline passed!';
33         ELSIF v_progress_pct >= 75              THEN v_status := 'ALMOST
34             THERE - Final push!';
35         ELSIF v_progress_pct >= 50              THEN v_status := 'HALFWAY
36             - Keep going!';
37         ELSIF v_progress_pct >= 25              THEN v_status := 'GOOD
38             START - Stay consistent!';
39         ELSE                                     v_status := 'JUST
40             STARTED - Build the habit!';
41
42         END IF;
43
44         DBMS_OUTPUT.PUT_LINE('--- Goal Tracker: ' || v_name || '
45             ---');
46
47         DBMS_OUTPUT.PUT_LINE('Goal      : ' || v_goal_type);
48         DBMS_OUTPUT.PUT_LINE('Target    : ' || v_target);
49         DBMS_OUTPUT.PUT_LINE('Progress : ' || v_progress_pct ||
50             '% of timeline');
51         DBMS_OUTPUT.PUT_LINE('Status   : ' || v_status);
52     END LOOP;
```

```
41     CLOSE goal_cur;  
42 END;  
43 /
```

Listing 8.2: Cursor 2 — goal progress tracker (cursors.sql)

8.3 Cursor 3 — Weekly Workout Streak Counter

Iterates over all users and, using a nested FOR loop and daily `SELECT COUNT(*)` queries, counts how many consecutive days (up to 7) each user has logged at least one workout.

```
1  DECLARE  
2      CURSOR streak_cur IS  
3          SELECT user_id, name FROM USERS;  
4  
5      v_user_id    USERS.user_id%TYPE;  
6      v_name       USERS.name%TYPE;  
7      v_streak     NUMBER := 0;  
8      v_check_date DATE;  
9      v_count      NUMBER;  
10  
11 BEGIN  
12     OPEN streak_cur;  
13     LOOP  
14         FETCH streak_cur INTO v_user_id, v_name;  
15         EXIT WHEN streak_cur%NOTFOUND;  
16  
17         v_streak      := 0;  
18         v_check_date := TRUNC(SYSDATE - 1);  
19  
20         FOR i IN 1..7 LOOP  
21             SELECT COUNT(*) INTO v_count  
22             FROM WORKOUTS  
23             WHERE user_id = v_user_id  
24             AND TRUNC(workout_date) = v_check_date;  
25  
26             IF v_count > 0 THEN  
27                 v_streak      := v_streak + 1;  
28                 v_check_date := v_check_date - 1;  
29             ELSE
```

```
30         EXIT;
31     END IF;
32 END LOOP;
33
34     DBMS_OUTPUT.PUT_LINE('--- Workout Streak: ' || v_name ||
35         ' ---');
36     DBMS_OUTPUT.PUT_LINE('Current Streak : ' || v_streak || '
37         day(s)');
38     IF v_streak = 0 THEN DBMS_OUTPUT.PUT_LINE('Message:
39         Start today!');
40     ELSIF v_streak >= 5 THEN DBMS_OUTPUT.PUT_LINE('Message:
41         Outstanding! Keep it up!');
42     ELSIF v_streak >= 3 THEN DBMS_OUTPUT.PUT_LINE('Message:
43         Push for 5 days!');
44     ELSE DBMS_OUTPUT.PUT_LINE('Message:
45         Aim for 3 in a row!');
46     END IF;
47 END LOOP;
48 CLOSE streak_cur;
49 END;
50 /
```

Listing 8.3: Cursor 3 — workout streak counter (cursors.sql)

Chapter 9

Frontend Description

9.1 Architecture Overview

The frontend is a **single-page application (SPA)** built with HTML5, CSS3, and vanilla JavaScript. It communicates with the Flask backend through `fetch()` calls to `localhost:5000/api`.

Component	Description
Login Screen	Dropdown populated from <code>GET /api/users</code> ; select name to log in as user or click Coach
Sidebar Navigation	Role-aware: User sees 6 personal tabs; Coach sees 10 tabs including analytics
User Dashboard	Personal profile card, city weather widget, recent workouts, active goals, recommendations
Coach Dashboard	All member cards, aggregated stats, latest workouts across all users, live weather panel
Data Entry Modals	User forms auto-fill <code>user_id</code> silently; Coach forms show user picker; IDs auto-assigned
Analytics (Coach)	4 view tabs and 6 query tabs rendered as dynamic tables
Toast Notifications	Success/error feedback on every insert operation

Table 9.1: Frontend component summary

9.2 Role-Based Access Control

- **User role:** All API calls include `?user_id={id}`, filtering data to only that user's records. Insert forms silently inject the logged-in user's ID. Record PKs are auto-fetched from `GET /api/next-id/{table}` using `MAX(pk)+1` from Oracle.
- **Coach role:** API calls return full datasets without filters. The coach selects the target user from a name dropdown. Exclusive access to Analytics Views and

Advanced Queries tabs.

9.3 REST API Endpoint Reference

Method	Endpoint	Description
GET	/api/users	All users with aggregated stats
POST	/api/users	Insert new user
GET	/api/users/{id}	Single user profile
GET	/api/workouts	All workouts (optional ?user_id=)
POST	/api/workouts	Log new workout
GET	/api/meals	All meals (optional ?user_id=)
POST	/api/meals	Log new meal
GET	/api/sleep	All sleep logs (optional ?user_id=)
POST	/api/sleep	Log sleep entry
GET	/api/goals	All goals (optional ?user_id=)
POST	/api/goals	Set new goal
GET	/api/recommendations	Recommendations (optional ?user_id=)
POST	/api/recommendations	Add recommendation
GET	/api/weather	Latest weather per city
POST	/api/weather	Insert weather record
GET	/api/next-id/{table}	Auto-fetch next primary key
GET	/api/view/{name}	Query analytics view
GET	/api/query/{name}	Run advanced query

Table 9.2: Complete REST API endpoint reference

Chapter 10

Screenshots

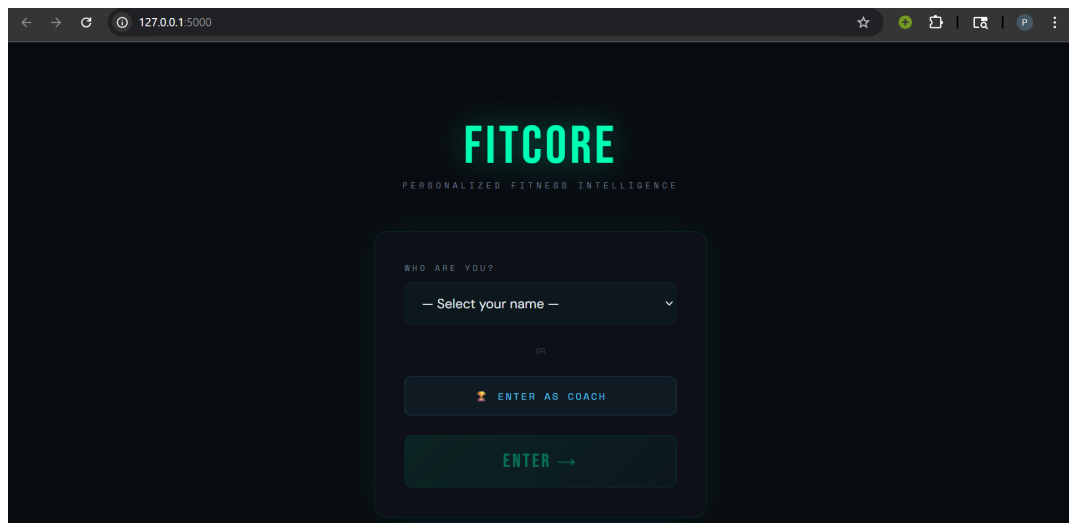


Figure 10.1: Login Screen — user name dropdown and Coach entry button

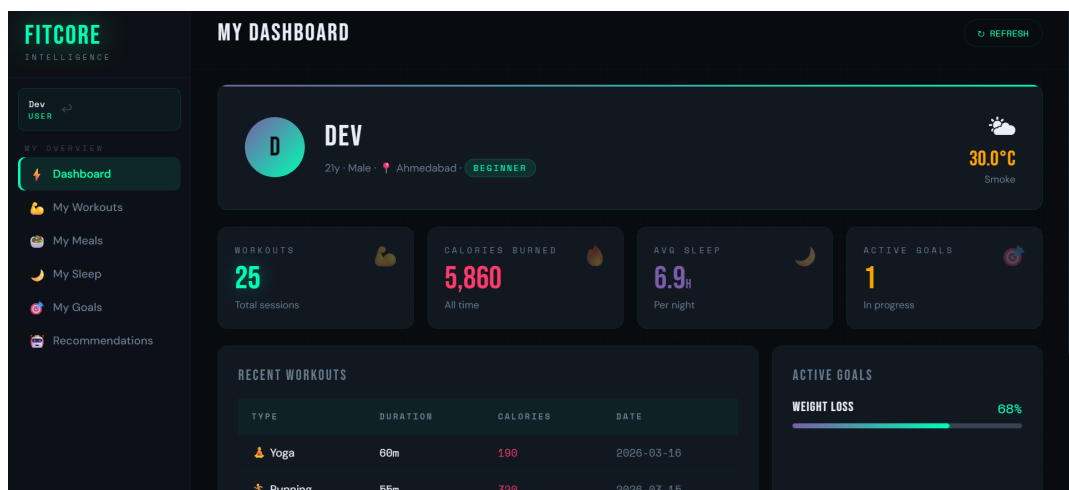


Figure 10.2: User Dashboard — personal profile card, city weather, and recent workouts

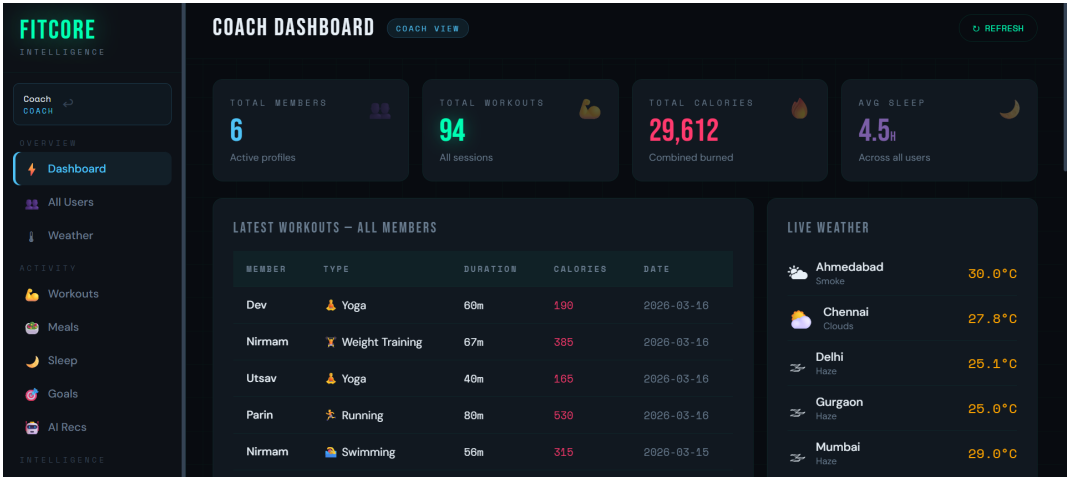


Figure 10.3: Coach Dashboard — all members overview with aggregated statistics

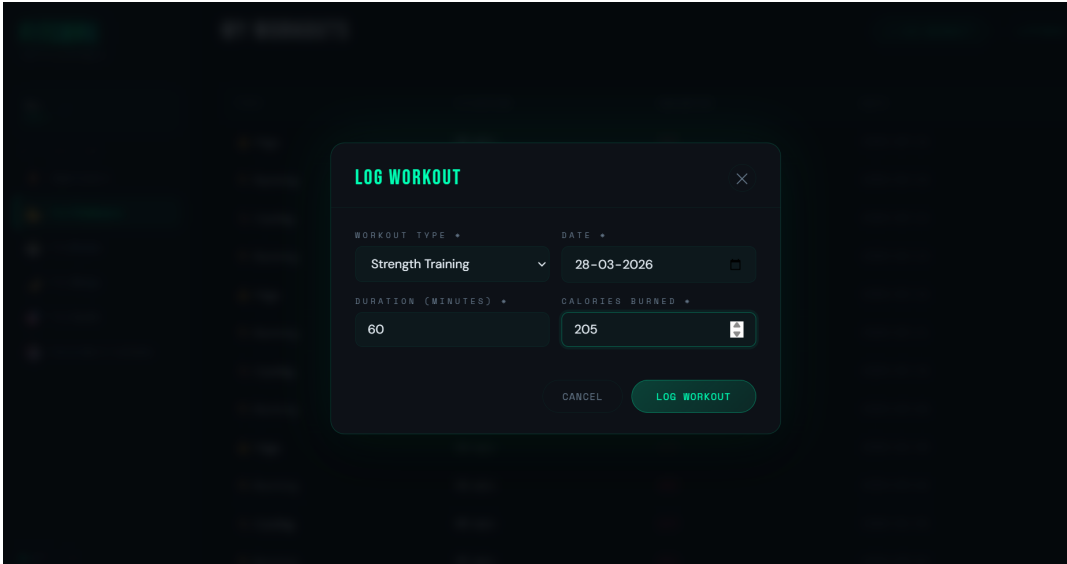


Figure 10.4: Log Workout modal — no manual ID fields, date pre-filled to today

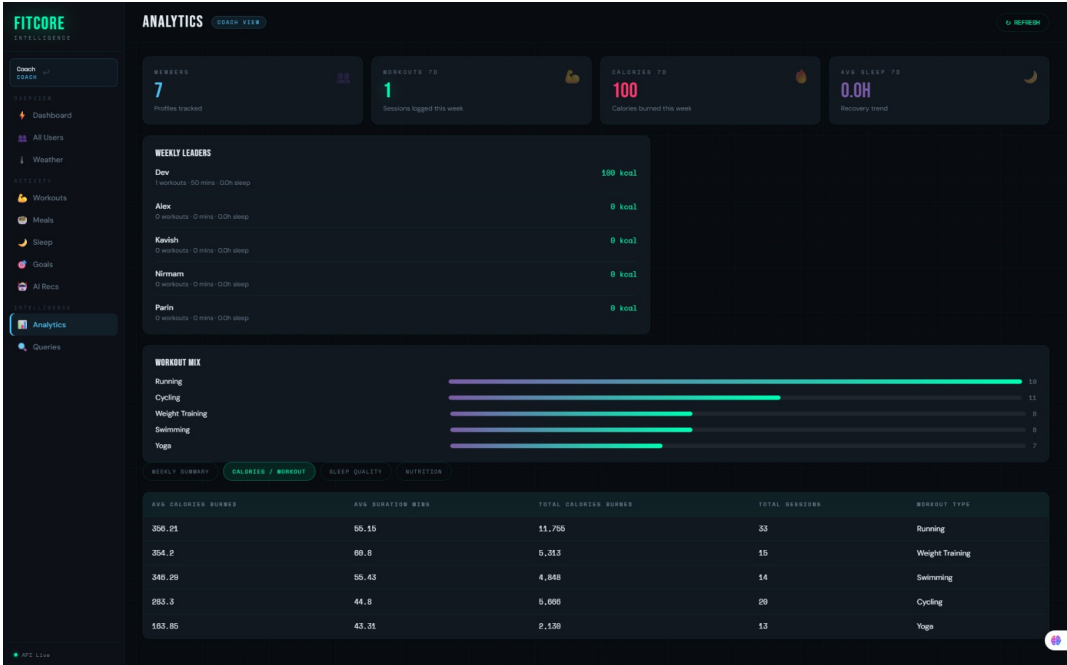


Figure 10.5: Coach Analytics — Advanced Queries tab showing Top Performers

Chapter 11

Conclusion & Future Scope

11.1 Conclusion

The Personalized Fitness Tracker successfully demonstrates a complete, working database application grounded in relational database principles. All stated objectives were met:

- **A fully normalised (3NF) schema** with seven interrelated tables captures all fitness data without redundancy or update anomalies.
- **Three PL/SQL explicit cursor blocks** implement intelligent daily recommendations, goal progress tracking, and workout streak analysis entirely at the database layer.
- **Five database views** abstract common aggregations, keeping the Flask API layer clean and readable.
- **Seven advanced queries** covering joins, aggregation, date arithmetic, and CASE expressions support the analytics dashboard.
- **A role-based frontend** gives users a focused personal experience while giving coaches complete operational visibility.
- **Real-time weather integration** connects live OpenWeatherMap data directly to the PL/SQL recommendation logic.

11.2 Future Scope

1. **Authentication:** Replace dropdown login with proper password-based authentication with hashed passwords stored in Oracle.
2. **Scheduled Cursors:** Run the recommendation cursor block as a daily DBMS_SCHEDULER job automatically.

References

1. Oracle Corporation. *Oracle Database 21c SQL Language Reference*. 2023.
<https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/>
2. Oracle Corporation. *Oracle Database PL/SQL Language Reference*. 2023.
<https://docs.oracle.com/en/database/oracle/oracle-database/21/lpls/>
3. python-oracledb. *python-oracledb 2.x Documentation*.
<https://python-oracledb.readthedocs.io/>
4. Pallets Projects. *Flask Documentation 3.x*.
<https://flask.palletsprojects.com/>
5. OpenWeatherMap. *Current Weather Data API Documentation*.
<https://openweathermap.org/current>